

B. Classrooms

The new semester is about to begin, and finding classrooms for orientation activities is always a headache.

There are k classrooms on campus and n proposed activities that need to be assigned a venue. Every proposed activity has specific starting time s_i and ending time f_i . Any such an activity should take place at one of the classrooms. Any of the k classrooms is big enough to hold any of the proposed activities, and each classroom can hold at most one activity at any time. No two proposed activities can take place at the same classroom at the same time. Even if two proposed activities overlap momentarily (the ending time of one activity equals the starting time another activity), they cannot be assigned to the same classroom.

There are so many proposed activities that there may not be enough classrooms to hold all the activities. It is desirable to have as many activities as possible. At most how many proposed activities can be assigned to the classrooms?

Input

- The first line contains two positive integers n and k ($1 \leq k \leq n \leq 200\,000$), representing the number of proposed activities and number of classrooms, respectively.
- The following n lines each contains two positive integers: the i th line among these n lines contains s_i and f_i ($1 \leq s_i \leq f_i \leq 10^9$), indicating the starting time and ending time of proposed activity i .

Output

Output an integer indicating the maximum number proposed activities that can be scheduled.

Sample Input 1

4	2
1	4
2	9
4	7
5	8

Sample Output 1

3

Edit & Submit		Metadata	My Submissions		Hide >
JUDGEMENT		RUNTIME	LANGUAGE	TEST CASES	
Accepted	0.24 s	C++	21/21	View Details	
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓					
Time Limit Exceeded	> 3.00 s	C++	9/21	View Details	
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✗					
View All Submissions					

The question gives n activities(index i) with starting time s_i and finish time f_i and there are k classrooms. The task is to return the number of activities we can host using k classrooms without having 2 activities overlapping in the same classroom.

This is a greedy question. And greedy policy is consider those activites that end earliest and we put them into the valid classroom as late as we can.

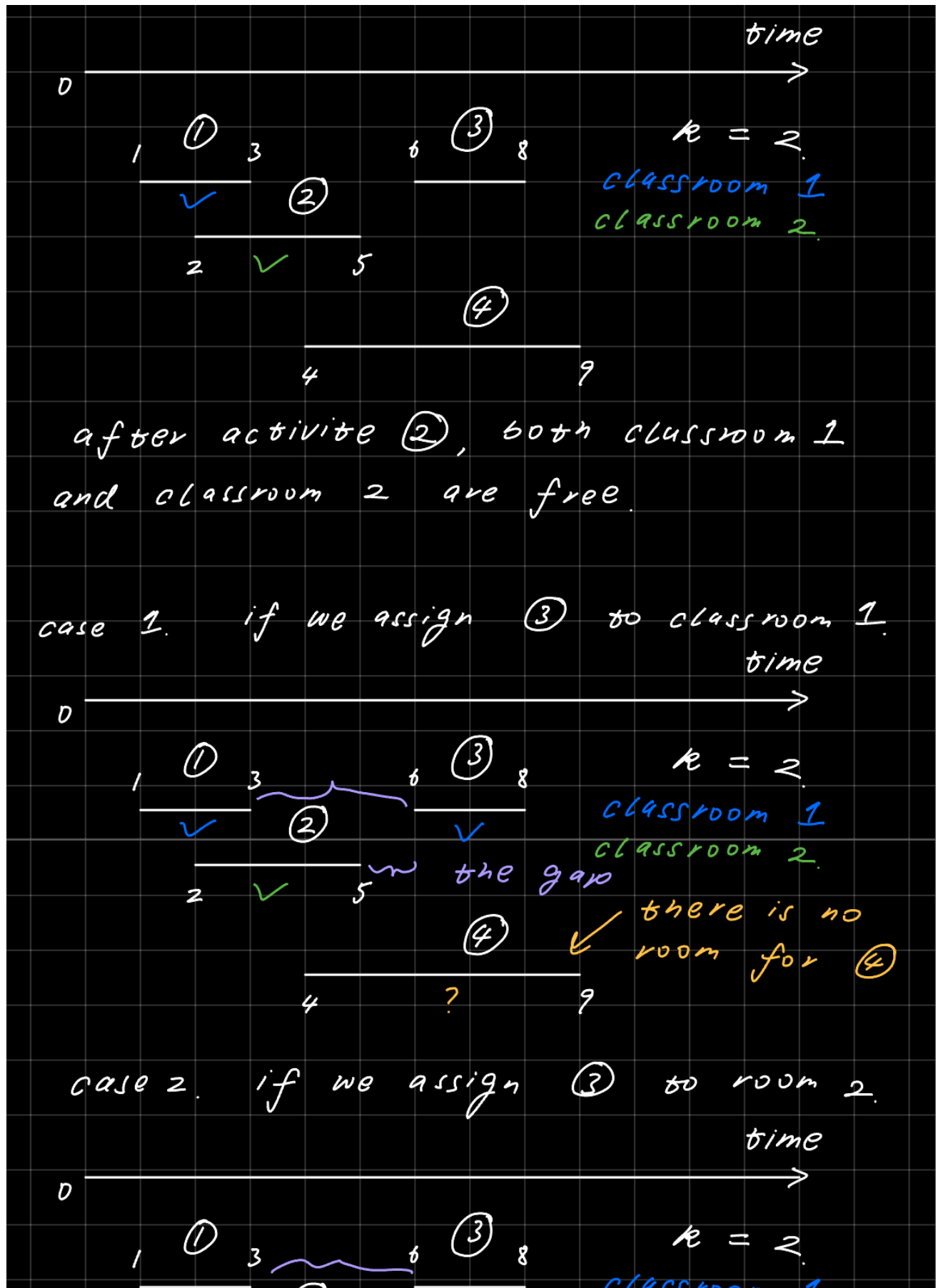
- ## 1. Why Earliest End Time?

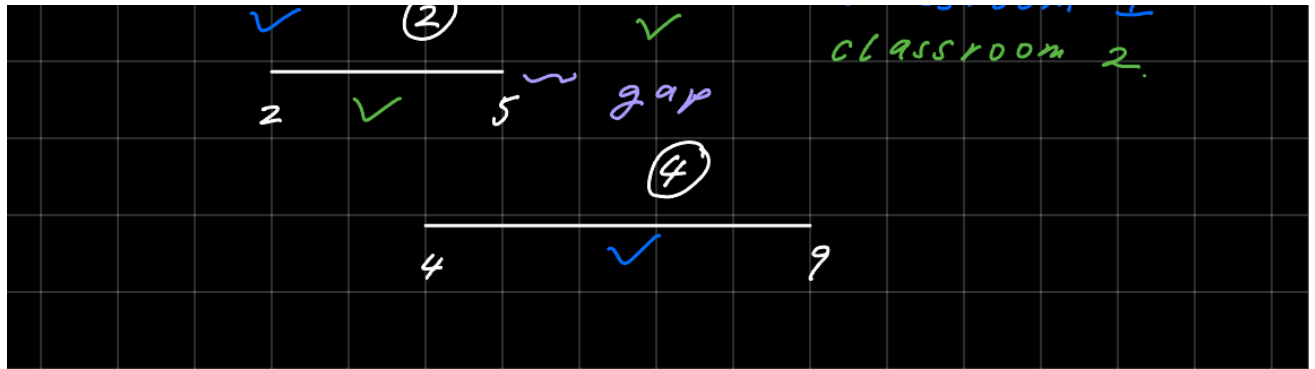
If we only have one single room, this question becomes the standard question of Interval Scheduling. The greedy policy is to arrange the activities that end earliest first, i.e. sort activities according to the ending time. A simple logic to explain is that the more early the current activity ends, the more time the rest activities can get which will lead to an overall optimal solution.

- ## 2. Why Latest Valid Classroom?

To be simple, the idea is we want to minimize the time gap between the end of previous activite and the start of the next activite for each classroom. This means for every vaild activite we have to assign it to the classroom that has previous activite ends latest.

Here is an example:





This example shows that by minimize the gap between the previous finishing time to the next starting time, we can allow more activites hosted by the same ammount of classrooms.

Challenges and Overcoming

```
int room_id = -1;
int latest_room_available_after = -2;
for (int j = 0; j < k; j++) {
    if (rooms_available_after[j] < start_time) {
        if (latest_room_available_after < rooms_available_after[j]) {
            room_id = j;
            latest_room_available_after = rooms_available_after[j];
        }
    }
}

if (room_id != -1) {
    rooms_available_after[room_id] = finish_time;
    result++;
}
```

I was using this loop to find the classroom that have previous activite ends latest, but this cause time limit exceeded. Instead of using this, a better approach is to use a multiset which stores the current finishing time of the last activite assigne to the room. We can use the lower_bound() method to find the first classroom that have finishing time larger than the starting time of current activite, and naturally the previous one is the room with the previous activite ends latest.